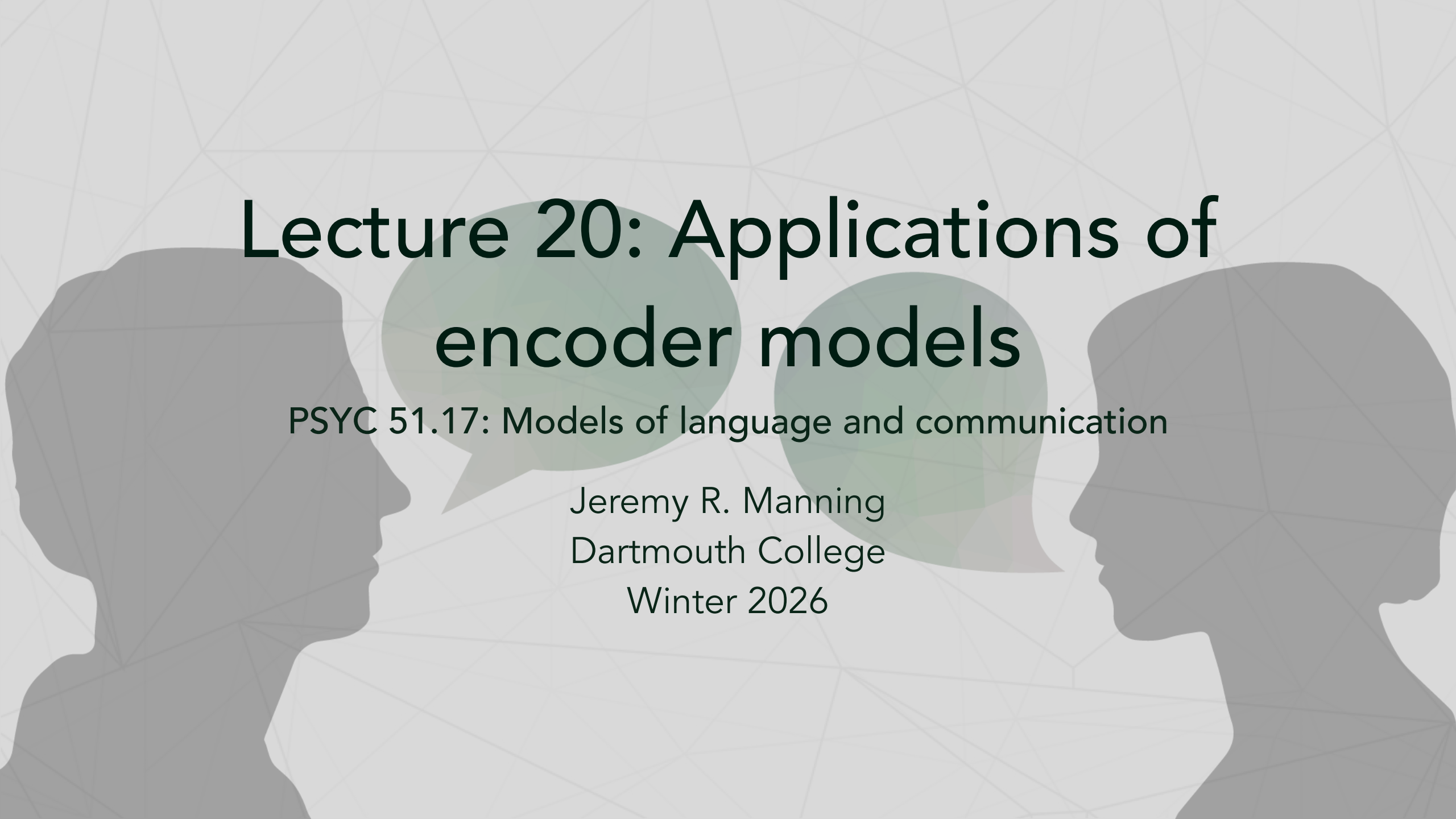




Lecture 20: Applications of encoder models

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Apply BERT to real-world NLP tasks: classification, NER, QA, and semantic similarity
2. Explain how BERT improved Google Search
3. Connect transformer representations to cognitive neuroscience findings
4. Evaluate the "understanding vs pattern matching" debate
5. Identify limitations and biases in deployed language models

Where BERT excels

BERT powers a wide range of understanding tasks

Classification tasks: Sentiment analysis, topic classification, spam detection, intent recognition

Token-level tasks: Named entity recognition (NER), part-of-speech tagging, word sense disambiguation

Span-level tasks: Question answering, extractive summarization, information extraction

Sentence-pair tasks: Semantic similarity, natural language inference, paraphrase detection

Industry impact

BERT powers Google Search (understanding queries), customer service chatbots, content moderation systems, and document understanding pipelines at companies worldwide.

Case study: Google Search

BERT revolutionized search in 2019

Before BERT, search engines primarily matched keywords. BERT enabled Google to understand *how words relate to each other* — especially prepositions, negations, and context words.

Why word order matters

```
1 query = "2019 brazil traveler to usa need a visa"
2
3 # Before BERT (keyword matching):
4 # Matches both "US traveler to Brazil" AND "Brazil traveler
  to US"
5
6 # With BERT (contextual understanding):
7 # Understands "to USA" means the traveler's DESTINATION is
  the US
8 # Correctly ranks: "Brazil citizen visa requirements for
  USA"
```

Question answering with BERT

Extractive QA: find the answer span in a passage

```
1 from transformers import pipeline
2
3 qa = pipeline("question-answering",
4               model="bert-large-uncased-whole-word-masking-
   finetuned-squad")
5
6 result = qa(
7     question="In what country is Normandy located?",
8     context="The Normans were the people who in the 10th and
   11th centuries "
9           "gave their name to Normandy, a region in France."
10 )
11 print(result)
12 # {'answer': 'France', 'score': 0.987, 'start': 134, 'end': 140}
```

How it works

BERT encodes the question and the passage into vectors, and then finds the best matching span in the passage.

Named entity recognition

Token-level classification with BERT

```
1 from transformers import pipeline
2
3 ner = pipeline("ner", model="dslim/bert-base-NER")
4
5 text = "Apple Inc. is headquartered in Cupertino, California."
6 entities = ner(text)
7
8 for e in entities:
9     print(f"{e['word']}: {e['entity']} (score:
10         {e['score']:.2f})")
11 # Apple: B-ORG (0.99)
12 # Inc: I-ORG (0.99)
13 # Cupertino: B-LOC (0.99)
14 # California: B-LOC (0.99)
```


Sentiment analysis

Sequence classification with BERT

```
1 from transformers import pipeline
2
3 sentiment = pipeline("sentiment-analysis",
4                       model="distilbert-base-uncased-finetuned-
5 sst-2-english")
6
7 texts = [
8     "This movie was absolutely amazing!",
9     "The product broke after one week.",
10    "The weather is cloudy today."
11 ]
12
13 for text in texts:
14     result = sentiment(text)[0]
15     print(f"{text}")
16     print(f" → {result['label']} (confidence: ")
```

Semantic similarity

Measuring sentence similarity with BERT embeddings

```
1 from transformers import BertTokenizer, BertModel
2 import torch.nn.functional as F
3
4 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
5 model = BertModel.from_pretrained('bert-base-uncased')
6
7 def get_sentence_embedding(sentence):
8     inputs = tokenizer(sentence, return_tensors='pt',
9 padding=True, truncation=True)
9     outputs = model(**inputs)
10    return outputs.last_hidden_state[:, 0, :] # [CLS] token
11
12 sent1 = "The cat is sleeping on the couch"
```

continued...

Semantic similarity

Measuring sentence similarity with BERT embeddings

```
13 sent2 = "A feline is resting on the sofa"      # Paraphrase
14 sent3 = "The weather is nice today"           # Unrelated
15
16 emb1, emb2, emb3 = [get_sentence_embedding(s) for s in [sent1,
17 sent2, sent3]]
18
18 print(f"Paraphrase similarity: {F.cosine_similarity(emb1,
19 emb2).item():.3f}") # High
19 print(f"Unrelated similarity: {F.cosine_similarity(emb1,
emb3).item():.3f}") # Low
```

...continued

Cognitive neuroscience perspective

How do brains and models process language?

Both biological brains and transformer models process language through **predictive processing** — constantly anticipating upcoming input based on context.

In the brain:

- The N400 ERP component reflects surprise at unexpected words
- Left inferior frontal gyrus (IFG) handles syntax
- Left superior temporal gyrus (STG/MTG) handles semantics
- Prediction errors drive learning and adaptation

In transformer models:

- Cross-entropy loss measures surprise at each token
- Lower layers encode syntax, upper layers encode semantics
- Gradient descent optimizes predictions
- Both systems are hierarchical, context-sensitive, and predictive

Prediction in brains vs language models

Parallels between neural and artificial systems

Phenomenon	Human brain	Transformer models
Surprise	N400 amplitude (EEG)	Cross-entropy loss
Hierarchy	Sounds → words → sentences	Tokens → phrases → meaning
Context	Prior discourse + world knowledge	Self-attention over sequence
Representation	Population coding (neurons)	Distributed embeddings (vectors)

The N400 / surprisal parallel

```
1 sentence_a = "I take my coffee with cream and sugar"    # Expected
2 sentence_b = "I take my coffee with cream and socks"    # Surprising!
3
4 # Brain: N400 amplitude much higher for "socks"
5 # Model: Higher cross-entropy loss for "socks"
6
7 surprisal = -np.log(model.predict_prob("socks", context))
8 # Surprisal correlates with N400 amplitude in EEG studies!
```

Neural encoding with language models

Can we predict brain activity from BERT representations?

```
1  # Neural encoding experiment workflow
2  import numpy as np
3  from transformers import BertModel
4
5  # 1. Participant reads sentences while in fMRI scanner
6  sentences = ["The dog chased the cat", "She opened the door",
7  ... ]
8  brain_activity = fmri_scanner.record(sentences) # (n_sentences,
9  n_voxels)
10
11 # 2. Extract BERT representations for same sentences
12 bert = BertModel.from_pretrained("bert-base-uncased")
```

continued...

Neural encoding with language models

Can we predict brain activity from BERT representations?

```
11 bert_embeddings = []
12 for sent in sentences:
13     outputs = bert(tokenizer(sent, return_tensors="pt"))
14     bert_embeddings.append(outputs.hidden_states[8].mean(dim=1)) #
    Layer 8
15
16 # 3. Train encoding model: BERT representations → brain activity
17 from sklearn.linear_model import Ridge
18 encoder = Ridge().fit(bert_embeddings[:80], brain_activity[:80])
19
20 # 4. Predict brain activity for held-out sentences
```

...continued...

Neural encoding with language models

Can we predict brain activity from BERT representations?

```
21 predictions = encoder.predict(bert_embeddings[80:])
22 correlation = np.corrcoef(predictions.flat,
23   brain_activity[80:].flat)[0,1]
# Correlation ~ 0.3–0.5 in language areas (statistically
# significant!)
```

...continued

Key finding: BERT layer 8 best predicts semantic brain areas; layers 2–4 predict phonological areas.

Understanding vs pattern matching

Questions to consider

Evidence FOR understanding:

- Captures syntax and semantics automatically
- Resolves lexical ambiguity based on context
- Handles long-range dependencies
- Generalizes to unseen examples
- Predicts brain activity patterns

Evidence AGAINST understanding:

- No grounding in the physical world
- No sensory or social experience
- Brittle to adversarial examples
- No common sense reasoning
- Might be "just" sophisticated pattern matching

The key question: Is there a meaningful difference between "understanding" and

Adversarial examples and brittleness

BERT can be fooled by simple tricks

```
1 from transformers import pipeline
2 classifier = pipeline("sentiment-analysis")
3
4 # Works correctly
5 classifier("This movie was absolutely wonderful!")
6 # → POSITIVE (0.9998)
7
8 # Adding irrelevant negative words flips the prediction!
9 classifier("This movie was absolutely wonderful! [SEP] bad bad
10 bad bad")
11 # → NEGATIVE (0.9234) # WRONG!
12
13 # Synonym substitution changes confidence
14 classifier("The food was good") # → POSITIVE (0.99)
15 classifier("The food was fine") # → POSITIVE (0.72) # Less
16 confident
```

Bias in language models

Models reflect and amplify societal biases

BERT inherits biases from its training data (books and Wikipedia contain historical biases):

- Gender bias in occupations: "The doctor said [MASK] would be late" → "he" (62%) vs "she" (18%)
- "The nurse said [MASK] would be late" → "she" (71%) vs "he" (15%)
- Racial bias in sentiment: Different confidence scores for identical sentences with different names

Mitigation strategies

1. **Data-level:** Balanced training corpora, counterfactual data augmentation
2. **Model-level:** Debiasing loss functions, fine-tuning on balanced datasets
3. **Output-level:** Post-hoc filtering, human review for sensitive applications
4. **Evaluation:** Regular bias audits using standardized benchmarks (WinoBias, StereoSet)

Limitations of current encoder models

Despite impressive performance, significant limitations remain

Quadratic complexity: Self-attention scales as $O(n^2)$ with sequence length, limiting context to 512–4096 tokens.

No generation capability: BERT is designed for understanding, not producing text. For generation tasks, use decoder models (GPT) or encoder-decoder models (T5, BART).

Data hunger: Pre-training requires billions of words. Humans learn language with orders of magnitude less data.

No world model: BERT learns statistical patterns in text but has no grounding in physical reality, sensory experience, or causal reasoning.

Fragility: Small input perturbations can cause large output changes, making models unreliable in adversarial settings.

Practical deployment tips

Making BERT work in production

Start with pre-trained models — never train from scratch. Use HuggingFace Model Hub.

Fine-tuning best practices:

- Learning rate: 1e-5 to 5e-5 (much lower than pre-training)
- Warmup steps: 6–10% of total training steps
- Monitor for overfitting; freeze early layers if data is limited
- 3–5 epochs is usually sufficient

Optimization for speed:

- Use DistilBERT for 60% faster inference with 97% quality
- Quantization (INT8) gives 2–4× speedup with minimal quality loss
- Export to ONNX Runtime for production serving
- Use FlashAttention when available

Optimization results

Optimization	Size	Latency	Quality
Original (FP32)	420MB	50ms	100%
Quantized (INT8)	110MB	25ms	99.5%
DistilBERT (smaller)	150MB	10ms	97%

Encoder vs decoder: looking ahead

Different architectures for different tasks

	Encoder (BERT)	Decoder (GPT)
Training	Masked language modeling	Autoregressive next-token
Context	Bidirectional	Left-to-right (causal)
Best for	Understanding tasks	Generation tasks
Examples	Classification, NER, QA, similarity	Text completion, dialogue, creative writing

Interesting trend: Decoder-only models (GPT-3, LLaMA) can also do classification via prompting. "In-context learning" blurs the distinction between understanding and generation architectures. The field is moving toward unified decoder-only models.

Discussion

Questions to consider

1. **Understanding vs pattern matching:** Where do you draw the line? Is there a test for "true" understanding? Does it matter if the application works?
2. **Brain-model parallels:** How useful are comparisons between transformers and the brain? What can neuroscience learn from AI, and vice versa?
3. **Bias and fairness:** Who is responsible for addressing bias in language models? Can we ever have completely unbiased models? How do we balance accuracy and fairness?
4. **The future of encoder models:** Will BERT-style encoders remain relevant as GPT-style decoders get better at everything? Or do understanding and generation require fundamentally different

References

Further reading

Devlin et al. (2019, *NAACL*) "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding"

Caucheteux & King (2022, *Nature Communications*) "Brains and algorithms partially converge in natural language processing"

Kuperberg & Jaeger (2016, *Language, Cognition and Neuroscience*) "What do we mean by prediction in language comprehension?"

Bender & Koller (2020, *ACL*) "Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data"

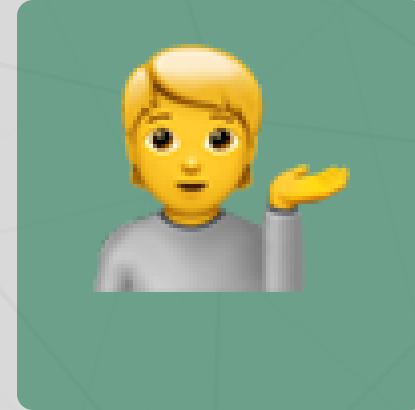
Questions?



Email



Discord



Office hours

Up next...

Week 7: GPT and autoregressive generation — from decoder architecture to text generation